

Làm Launcher Minecraft Java Điện Thoại

https://gemini.google.com/app/fd6b5c80de010a6a

User prompt: bjan ơi mình đã làm được cái launcher pc rồi giờ mình muốn làm cái app trên điệ thoại nhưng vì mình chưa bảo giờ tò mò đến nên cuungx chưa biết làm sao xài cái gì cả nhưng mà mình muốn làm java ấy không phía bedrock đâu

Response: Để chạy được **Minecraft: Java Edition** trên điện thoại (chủ yếu là Android), cơ chế hoạt động phức tạp hơn rất nhiều so với PC Launcher.

Trên PC, launcher chỉ cần tải game và gọi lệnh `java -jar ...`. Nhưng trên điện thoại, Android không dùng chuẩn JVM thông thường mà dùng ART (Android Runtime), và GPU điện thoại chỉ hỗ trợ OpenGL ES / Vulkan chứ không hỗ trợ đầy đủ Desktop OpenGL mà Minecraft Java yêu cầu.

Kiến trúc cốt lõi để chạy Minecraft Java trên Android

- OpenJDK ported cho Android (ARM/ARM64):** Cần một bản build OpenJDK (Java 8, 17, 21...) được biên dịch bằng NDK dành riêng cho kiến trúc chip di động (aarch64/armv7a).
- Graphics Translation Layer (Bộ dịch đồ họa):** Minecraft Java gọi các hàm Desktop OpenGL. Bạn phải dịch các lệnh này sang **OpenGL ES** hoặc **Vulkan** để GPU điện thoại hiểu được thông qua các thư viện như:
 - GL4ES:** Dịch OpenGL 2.1/1.x sang OpenGL ES 2.0/3.0.
 - Zink / Mesa / Angle:** Dịch Desktop OpenGL sang Vulkan (hiệu năng cao cho các bản Minecraft mới).
- LWJGL & GLFW Wrapper (Native Bridge):** Minecraft dùng LWJGL để xử lý cửa sổ, âm thanh (OpenAL) và đầu vào. Trên Android, bạn phải viết JNI/C++ wrapper để bắt các sự kiện từ Android SurfaceView/NativeActivity chuyển vào LWJGL.
- Hệ thống điều khiển ảo (Touch Controls):** Giao diện nút bấm ảo trên màn hình cảm ứng để gửi tín hiệu chuột/bàn phím vào cửa sổ game.

Các dự án mã nguồn mở tham khảo tốt nhất

Dự án	Ngôn ngữ / Công nghệ	Điểm nổi bật
PojavLauncher (Khuyến dùng)	Java, C/C++ (JNI), OpenJDK, GL4ES	Dự án hoàn thiện và nổi tiếng nhất thế giới về chạy Minecraft Java trên Android/iOS.
Fold Craft Launcher (FCL)	Kotlin, Java, C++	Giao diện hiện đại (Material You), tối ưu hóa tốt và cấu trúc code rất sạch.

Lộ trình tiếp cận cho bạn

- Hiểu cách PojavLauncher hoạt động:**
 - Tải mã nguồn của **PojavLauncher** hoặc **FCL** trên GitHub về đọc cấu trúc module.
 - Xem cách họ tổ chức thư mục JRE runtime (`libopenjdk.so`) và các thư viện native `.so` (GL4ES, LWJGL stub).
- Chọn hướng phát triển:**
 - Cách 1 (Tùy biến từ Base có sẵn):** Fork và tùy biến lại giao diện, tính năng từ PojavLauncher Core/FCL. Đây là cách khả thi nhất vì tự viết lại toàn bộ tầng C++ Bridge/Graphic Translator từ đầu mất hàng tháng đến hàng năm.
 - Cách 2 (Xây UI Launcher riêng + Gọi Core):** Viết app giao diện quản lý phiên bản/tài khoản/mod bằng Kotlin/Jetpack Compose, sau đó tích hợp core backend của Pojav để khởi chạy engine game.

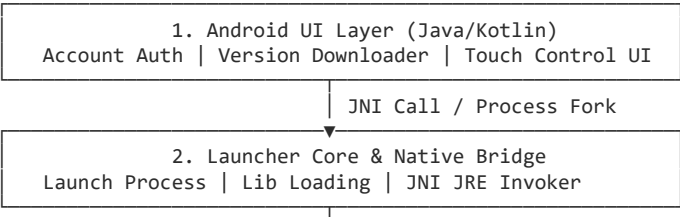
User prompt: Giải thích chi tiết kiến trúc và cách tổ chức mã nguồn của PojavLauncher trên Android.

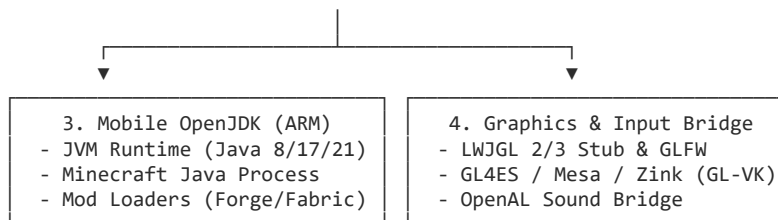
Response: Kiến trúc của **PojavLauncher** trên Android là sự kết hợp giữa **Android SDK UI**, **Native C/C++ Bridge**, và một môi trường **JVM (OpenJDK di động)** chạy độc lập.

Dưới đây là sơ đồ chi tiết về kiến trúc hoạt động và cách tổ chức mã nguồn của dự án.

1. Kiến trúc tổng thể (Architecture Overview)

Toàn bộ hệ thống được chia làm 4 tầng chính:





2. Chi tiết các thành phần cốt lõi

A. Khởi tạo JVM (Java Invocation API)

- PojavLauncher giải nén một bản runtime OpenJDK (ARM32 / AArch64) vào bộ nhớ trong của ứng dụng.
- Thay vì chạy `java -jar` qua terminal shell bị hạn chế trên Android, Pojav sử dụng file native C/C++ (thường gọi là `mojoexec` hoặc `jre_bridge`).
- Mã nguồn native sẽ gọi trực tiếp thư viện `libjvm.so` thông qua hàm `JNI_CreateJavaVM()` để khởi tạo máy ảo Java ngay trong một tiến trình mới, sau đó load class chính `net.minecraft.client.main.Main`.

B. Graphics Translation (Chuyển đổi đồ họa OpenGL)

Minecraft Java gọi Desktop OpenGL, trong khi Android chỉ hỗ trợ **OpenGL ES** hoặc **Vulkan**:

- GL4ES / NG-GL4ES:** Thư viện wrapper dịch các tập lệnh Desktop OpenGL (1.x, 2.1) sang OpenGL ES 2.0/3.0.
- Zink / Mesa / MobileGlues:** Dành cho các phiên bản Minecraft mới hơn (1.17+) và Shader. Công nghệ này dịch các lệnh OpenGL sang Vulkan API nhằm đạt hiệu năng tối đa.
- SurfaceView Binding:** Tầng native bind framebuffer của Minecraft vào `SurfaceView` của Android để hiển thị lên màn hình.

C. LWJGL & GLFW Emulation (Cửa sổ & Đầu vào)

- Minecraft dựa vào thư viện **LWJGL 2 / LWJGL 3** và **GLFW** để mở cửa sổ, nhận tín hiệu chuột/phím và xuất âm thanh.
- PojavLauncher build lại bản custom **LWJGL stub**. Các hàm như `glfwCreateWindow`, `glfwPollEvents` được trỏ tới các hàm native của launcher để nhận sự kiện chạm từ màn hình cảm ứng Android rồi giả lập thành tọa độ chuột và keycode bàn phím gửi vào game.

3. Cấu trúc thư mục mã nguồn PojavLauncher

Mã nguồn PojavLauncher được phân chia thành các module rõ ràng:

```

PojavLauncher/
├── app_pojavlauncher/                                # Module chính (Android App)
│   ├── src/main/
│   │   ├── java/net/kdt/pojavlauncher/
│   │   │   ├── authenticator/                        # Quản lý tài khoản (Microsoft / Offline)
│   │   │   ├── customcontrols/                       # Trình thiết kế & hiển thị phím ảo (Touch UI)
│   │   │   ├── fragments/                            # Giao diện cài đặt, danh sách phiên bản
│   │   │   ├── tasks/                                # Tải assets, thư viện, jar game từ Mojang
│   │   │   ├── utils/                                # Giải nén runtime, xử lý JSON manifest
│   │   │   ├── JREUtils.java                         # Quản lý các biến môi trường Java
│   │   │   └── MainActivity.java                      # Activity launcher & chuyển cảnh vào game
│   │   ├── cpp/
│   │   │   └── ...
│   │   │       # Mã nguồn C/C++ JNI
│   │   └── res/                                       # Các bridge kết nối giữa Android và native
│   └── build.gradle                                  # Layout XML, icons, drawable
├── forge_installer/                                # Tool hỗ trợ cài đặt Modloader (Forge, NeoForge, Fabric)
├── MobileGlues / NG-GL4ES                          # Submodule chuyển đổi OpenGL sang GLES/Vulkan
├── glfw / sdl                                       # Submodule giả lập cửa sổ và quản lý surface
├── mojoexec                                         # Module native chịu trách nhiệm nạp libjvm và exec process
└── scripts/                                         # Script build OpenJDK, sync ngôn ngữ
  
```

4. Luồng chạy của một phiên chơi game (Execution Flow)

1. Chuẩn bị dữ liệu (Java Layer):

- Người dùng nhấn **Play**.
- App đọc file `.json` của phiên bản (ví dụ: `1.20.4.json`), tải file client `.jar` và toàn bộ thư viện dependencies từ Mojang.

2. Khởi tạo môi trường (Native Layer):

- App thiết lập các biến môi trường: JAVA_HOME, LD_LIBRARY_PATH (trở tới thư mục chứa các file .so của runtime và translator).
- Khởi chạy Activity chứa game surface.

3. Boot Game (JNI Layer):

- Mã C++ gọi JNI_CreateJavaVM(), nạp các tham số JVM (-Xmx, -cp, ...).
- JVM chạy class net.minecraft.client.main.Main.

4. Game Loop:

- Game render frame -> Chuyển qua GL4ES/Zink -> Render lên Android Surface.
- Người dùng chạm vào nút ảo -> CustomControls convert thành event -> GLFW stub -> Truyền vào Minecraft.

Lời khuyên khi bắt đầu thực hành

- **Không nên viết lại từ số 0:** Toàn bộ phần compiler OpenJDK và Graphics Wrapper là các bài toán kỹ thuật rất phức tạp.
- **Nên bắt đầu bằng việc clone PojavLauncher:** Đọc hiểu luồng tải game trong package net.kdt.pojavlauncher.tasks và cách cấu hình phím trong customcontrols, sau đó thử nghiệm thay đổi giao diện (UI) hoặc tùy biến tính năng download theo ý bạn.

Sources

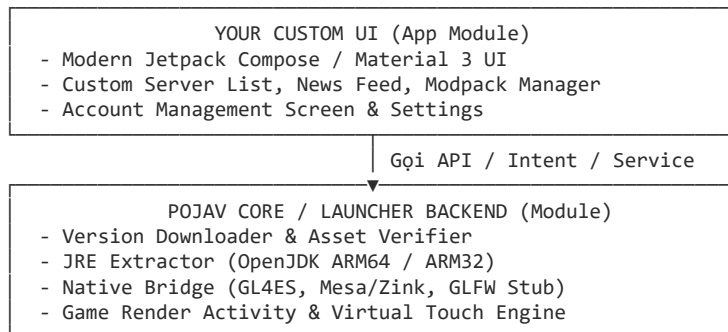
1. [PojavLauncher/README.md at v3_openjdk - GitHub](#)
2. [GitHub - TeamPojavLauncher/PojavLauncher: A Minecraft](#)

User prompt: Hướng dẫn cách viết UI launcher Android riêng biệt và tích hợp backend/core của PojavLauncher.

Response: Để viết một UI Launcher Android tùy biến (bằng **Kotlin** hoặc **Jetpack Compose**) và tận dụng toàn bộ sức mạnh backend (tải game, OpenJDK, Graphics bridge) của PojavLauncher, bạn nên áp dụng mô hình **Modular Architecture**.

Thay vì viết lại toàn bộ tầng Native C++ cực kỳ phức tạp, bạn sẽ tách PojavLauncher thành một **Core Library (AAR / Library Module)** và xây dựng giao diện Launcher riêng làm tầng Consumer.

Mô hình kiến trúc tích hợp



1 Thiết lập Project và Tách Core Module

Tổ chức mã nguồn thành dạng Multi-Module

1. Clone mã nguồn của PojavLauncher từ GitHub:

```
git clone --recurse-submodules https://github.com/PojavLauncherTeam/PojavLauncher.git
```

2. Đổi module app_pojavlauncher từ com.android.application thành com.android.library trong build.gradle (hoặc export các package cốt lõi: authenticator, tasks, customcontrols, JNI bridges).

3. Tạo module giao diện mới của bạn (ví dụ :custom_ui) với plugin com.android.application, sau đó khai báo dependency:

```
dependencies {
    implementation project(':app_pojavlauncher')
    // Thêm Jetpack Compose hoặc XML dependencies cho UI mới
}
```

2 Viết logic Tải và Xác thực Phiên bản

Tận dụng Task system có sẵn của Pojav

Gọi các Task nền của Pojav để tải version manifest và game assets từ Mojang:

```
// Gọi tải Metadata danh sách phiên bản
val versionListTask = VersionListDownloadTask()
versionListTask.setListener { versions ->
    // Render danh sách phiên bản lên UI của bạn (Compose List / RecyclerView)
}
versionListTask.execute()

// Khi người dùng bấm Play, chạy chuỗi chuẩn bị tài nguyên
fun prepareAndDownloadGame(versionId: String) {
    val downloadTask = GameDownloadTask(versionId, object : DownloadCallback {
        override fun onProgress(progress: Int, max: Int) {
            // Cập nhật ProgressBar trên giao diện của bạn
        }
        override fun onComplete() {
            // Sẵn sàng để khởi chạy game
            launchMinecraftProcess(versionId)
        }
    })
    downloadTask.execute()
}
```

3 Cấu hình Môi trường Runtime & Java VM

Giải nén JRE và thiết lập Native Libraries

Đảm bảo OpenJDK và các file .so đồ họa đã được giải nén vào thư mục nội bộ (data/data/<package_name>/files/):

```
fun setupRuntimeEnvironment(context: Context) {
    // Kiểm tra và giải nén runtime Java (Java 17/21 cho bản mới, Java 8 cho bản cũ)
    val jreDir = File(context.filesDir, "runtime")
    if (!jreDir.exists()) {
        JREUtils.extractRuntime(context, "jre-17-arm64")
    }

    // Khởi tạo các thư viện đồ họa (GL4ES / Zink renderer)
    Tools.extractNativeLibs(context)
}
```

4 Kích hoạt Game Surface Activity

Chuyển quyền điều khiển sang SurfaceView & Touch Controller

Sau khi dữ liệu tải xong, UI Launcher của bạn sẽ gọi MainActivity (hoặc GameActivity của Pojav) thông qua Intent kèm theo các thông số khởi chạy:

```
fun launchMinecraftProcess(versionId: String) {
    val launchIntent = Intent(context, PojavGameActivity::class.java).apply {
        putExtra("EXTRA_VERSION_ID", versionId)
        putExtra("EXTRA_AUTH_TOKEN", currentAccount.accessToken)
        putExtra("EXTRA_AUTH_UUID", currentAccount.uuid)
        putExtra("EXTRA_USERNAME", currentAccount.username)
        putExtra("EXTRA_RENDERER", "GL4ES_1.1.5") // hoặc "ZINK_MESA"
        putExtra("EXTRA_RAM_MB", 2048)
        putExtra("EXTRA_JVM_ARGS", "-XX:+UseG1GC")
    }
    context.startActivity(launchIntent)
}
```

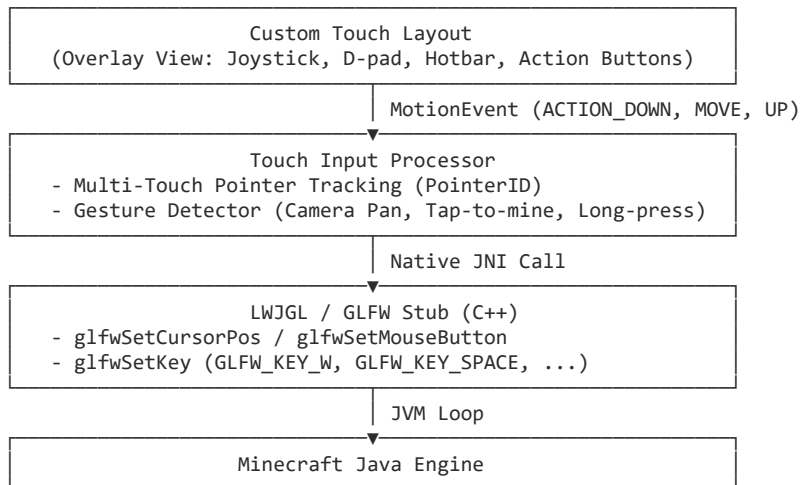
Một số lưu ý kỹ thuật quan trọng

- **Quản lý NDK và Architecture:** Luôn build với ABI arm64-v8a để tối ưu cho các máy đời mới, và giữ armeabi-v7a nếu muốn hỗ trợ thiết bị 32-bit cũ.
- **Bộ nhớ RAM:** Android quản lý RAM rất khắt khe. Hãy thêm thuộc tính android:largeHeap="true" vào AndroidManifest.xml để tránh crash do tràn bộ nhớ Java Heap.
- **Quyền truy cập lưu trữ:** Các phiên bản Android mới (Android 11+) yêu cầu quyền MANAGE_EXTERNAL_STORAGE nếu bạn muốn người dùng dễ dàng import mod, resource pack từ thư mục ngoài vào game.

User prompt: Hướng dẫn cách xây dựng và tối ưu hệ thống phím cảm ứng ảo (Touch Controls) cho Minecraft Java trên Android.

Response: Xây dựng hệ thống phím cảm ứng ảo (Touch Controls) cho Minecraft Java trên Android là bài toán chuyển đổi các cử chỉ chạm đa điểm (Multi-touch) thành các sự kiện chuột và phím của máy tính (LWJGL / GLFW Key & Mouse Events).

1. Kiến trúc hệ thống Touch Controls



1 Xây dựng Overlay View & Render Nút bấm

Tạo Custom View đè lên trên SurfaceView của Game

Tạo một `FrameLayout` hoặc `CustomView` nằm đè lên trên game surface. Định nghĩa cấu trúc dữ liệu cho mỗi nút ảo để dễ dàng lưu/tải từ JSON:

```

data class ControlButton(
    val id: String,
    val text: String,
    val keycode: Int,           // GLFW Keycode (ví dụ: GLFW_KEY_SPACE = 32)
    var x: Float,              // Tọa độ tương đối (%)
    var y: Float,
    var width: Float,
    var height: Float,
    var isToggleable: Boolean = false,
    var isPressed: Boolean = false
)
  
```

2 Xử lý Multi-touch và Touch Tracking

Phân tách từng ngón tay để không bị loạn khi bấm nhiều phím

Ghi đè hàm `onTouchEvent` và theo dõi từng điểm chạm theo `pointerId`:

```

override fun onTouchEvent(event: MotionEvent): Boolean {
    val action = event.actionMasked
    val pointerIndex = event.actionIndex
    val pointerId = event.getPointerId(pointerIndex)
    val touchX = event.getX(pointerIndex)
    val touchY = event.getY(pointerIndex)

    when (action) {
        MotionEvent.ACTION_DOWN, MotionEvent.ACTION_POINTER_DOWN -> {
            handlePointerDown(pointerId, touchX, touchY)
        }
        MotionEvent.ACTION_MOVE -> {
            // Duyệt qua tất cả các pointer đang chạm
            for (i in 0 until event.pointerCount) {
                val pId = event.getPointerId(i)
                handlePointerMove(pId, event.getX(i), event.getY(i))
            }
        }
        MotionEvent.ACTION_UP, MotionEvent.ACTION_POINTER_UP, MotionEvent.ACTION_CANCEL -> {
            handlePointerUp(pointerId)
        }
    }
    return true
}
  
```

3 Xử lý Joystick và Camera Pan (Xoay góc nhìn)

Chia màn hình thành vùng di chuyển và vùng xoay camera

Chia màn hình làm 2 vùng riêng biệt:

- **Nửa trái (Movement):** Đặt Virtual Joystick hoặc D-pad (gửi phím W, A, S, D).
- **Nửa phải (Camera):** Nhận sự kiện vuốt để tính độ chênh lệch (Δx , Δy) và gửi vào con trỏ chuột của game:

```
private fun handleCameraDrag(deltaX: Float, deltaY: Float, sensitivity: Float = 1.2f) {
    val mouseDx = deltaX * sensitivity
    val mouseDy = deltaY * sensitivity

    // Gửi delta di chuyển vào tầng C++ GLFW Stub
    NativeBridge.sendMouseMove(mouseDx, mouseDy)
}
```

4 JNI Bridge gửi sự kiện sang JVM Minecraft

Gọi hàm C++ để truyền tín hiệu thẳng vào LWJGL

Viết JNI Bridge (C++) kết nối từ Android sang GLFW của game:

```
#include <jni.h>
#include "glfw_stubs.h"

extern "C" {
    JNIEXPORT void JNICALL
    Java_net_kdt_pojavlaunch_NativeBridge_sendKeyEvent(JNIEnv* env, jclass clazz, jint keycode, jboolean isDown) {
        // Giả lập sự kiện bấm phím trong GLFW
        glfwInjectKey(keycode, isDown ? GLFW_PRESS : GLFW_RELEASE);
    }

    JNIEXPORT void JNICALL
    Java_net_kdt_pojavlaunch_NativeBridge_sendMouseButton(JNIEnv* env, jclass clazz, jint button, jboolean isDown) {
        // 0: Chuột trái (đặt block), 1: Chuột phải (đặt block/dùng đồ)
        glfwInjectMouseButton(button, isDown ? GLFW_PRESS : GLFW_RELEASE);
    }

    JNIEXPORT void JNICALL
    Java_net_kdt_pojavlaunch_NativeBridge_sendMouseMove(JNIEnv* env, jclass clazz, jfloat dx, jfloat dy) {
        glfwInjectCursorDelta(dx, dy);
    }
}
```

2. Các kỹ thuật tối ưu hóa độ nhạy và hiệu năng

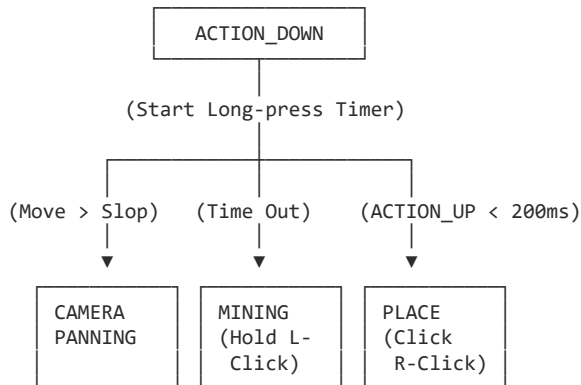
- **Giảm thiểu Garbage Collection (Zero Allocation trong onTouchEvent):** Tránh tạo đối tượng mới (ví dụ `new PointF()`) bên trong `onTouchEvent` hoặc vòng lặp `ACTION_MOVE`. Cấp phát tĩnh trước và tái sử dụng bộ nhớ để tránh hiện tượng khựng hình (micro-stutter).
- **Độ trễ phản hồi (Input Latency):** Không dùng `View.invalidate()` toàn màn hình khi chỉ cần cập nhật trạng thái phím bấm. Hãy vẽ các nút ảo bằng phần cứng qua `HardwareLayer` hoặc một layer `SurfaceView` phụ trong suốt.
- **Xử lý Tap vs Drag (Đặt block vs Đổi góc nhìn):**
 - Khi người dùng chạm nửa phải màn hình và giữ yên trong khoảng thời gian ngắn (> 300ms) mà không di chuyển quá một ngưỡng (Threshold $\approx 10px$), kích hoạt sự kiện **Giữ chuột trái (Mining)**.
 - Nếu di chuyển vượt ngưỡng -> Hủy trạng thái Mining và chuyển sang **Camera Pan**.
- **Đệm dữ liệu di chuyển chuột (Mouse Smoothing):** Tốc độ quét màn hình cảm ứng di động (60Hz/120Hz/240Hz) có thể không trùng khớp với FPS của game. Hãy dùng bộ lọc trung bình động (Moving Average) hoặc nội suy tuyến tính (Linear Interpolation - Lerp) để góc nhìn quay mượt mà, không bị giật.

User prompt: Hướng dẫn cách viết thuật toán *GestureDetector* phân biệt chạm đặt block và giữ lâu để đặt block trong Minecraft Java trên Android.

Response: Để mang lại trải nghiệm giống bản Bedrock trên điện thoại, vùng điều khiển camera (nửa phải màn hình) cần một bộ xử lý cử chỉ (**Gesture Detector**) với máy trạng thái (FSM - Finite State Machine) để phân tách rõ ràng 3 hành vi:

1. **Camera Pan (Xoay góc nhìn):** Ngón tay di chuyển vượt quá ngưỡng khoảng cách (*Touch Slop*).
2. **Tap / Short Click (Đặt block / Dùng đồ):** Chạm nhanh rồi nhấc tay lên trong thời gian ngắn (*Right Click*).
3. **Long Press / Hold (Đặt block / Đào quặng):** Giữ ngón tay đứng yên quá thời gian chờ rồi duy trì nhấn (*Hold Left Click*).

Máy trạng thái cử chỉ (Gesture State Machine)



1 Khởi tạo hằng số và trạng thái

Định nghĩa các thông số đo lường cử chỉ

Khai báo các biến trạng thái và ngưỡng phát hiện (Thresholds) để tái sử dụng trong vòng lặp cảm ứng:

```
enum class TouchState { IDLE, PENDING, PANNING, MINING }
```

```
class IngameGestureDetector(private val context: Context) {
    // Ngưỡng phát hiện
    private val touchSlop = ViewConfiguration.get(context).scaledTouchSlop.toFloat() // ~8-16px
    private val longPressTimeout = 250L // Thời gian chờ để kích hoạt đào (ms)
    private val tapTimeout = 200L // Thời gian tối đa của một cú chạm ngắn

    // Quản lý trạng thái
    private var state = TouchState.IDLE
    private var activePointerId = MotionEvent.INVALID_POINTER_ID
    private var startX = 0f
    private var startY = 0f
    private var lastX = 0f
    private var lastY = 0f
    private var downTime = 0L

    private val handler = Handler(Looper.getMainLooper())
    private val miningRunnable = Runnable {
        if (state == TouchState.PENDING) {
            state = TouchState.MINING
            // Kích hoạt nhấn giữ chuột trái (GLFW Mouse 0 Down)
            NativeBridge.sendMouseButton(0, true)
        }
    }
}
```

2 Xử lý ACTION_DOWN

Bắt đầu đếm giờ và ghi nhớ tọa độ gốc

Khi ngón tay chạm xuống, đặt trạng thái là PENDING và kích hoạt bộ đếm thời gian:

```
fun handleTouchDown(event: MotionEvent, pointerIndex: Int) {
    activePointerId = event.getPointerId(pointerIndex)
    startX = event.getX(pointerIndex)
    startY = event.getY(pointerIndex)
    lastX = startX
    lastY = startY
    downTime = System.currentTimeMillis()

    state = TouchState.PENDING
    // Lên lịch kích hoạt đào sau 250ms
    handler.postDelayed(miningRunnable, longPressTimeout)
}
```

3 Xử lý ACTION_MOVE

Kiểm tra ngưỡng di chuyển để chuyển sang xoay Camera

Tính toán khoảng cách dịch chuyển $d = \sqrt{\Delta x^2 + \Delta y^2}$. Nếu vượt quá touchSlop, hủy chế độ chờ đào và chuyển sang chế độ Camera:

```
fun handleTouchMove(event: MotionEvent) {
    val pointerIndex = event.findPointerIndex(activePointerId)
```

```

if (pointerIndex == -1) return

val currentX = event.getX(pointerIndex)
val currentY = event.getY(pointerIndex)
val deltaX = currentX - lastX
val deltaY = currentY - lastY

when (state) {
    TouchState.PENDING -> {
        val totalDistance = Math.hypot((currentX - startX).toDouble(), (currentY - startY).toDouble()).toFloat()
        if (totalDistance > touchSlop) {
            // Ngón tay đã di chuyển -> Hủy đếm giờ đào block
            handler.removeCallbacks(miningRunnable)
            state = TouchState.PANNING
            NativeBridge.sendMouseMove(deltaX, deltaY)
        }
    }
    TouchState.PANNING -> {
        // Tiếp tục gửi độ lệch chuột vào game
        NativeBridge.sendMouseMove(deltaX, deltaY)
    }
    TouchState.MINING -> {
        // Đang đào nhưng vẫn cho phép xoay camera nhẹ theo ngón tay
        NativeBridge.sendMouseMove(deltaX * 0.5f, deltaY * 0.5f)
    }
    TouchState.IDLE -> {}
}

lastX = currentX
lastY = currentY
}

```

4 Xử lý ACTION_UP / ACTION_CANCEL

Phát lệnh Đặt block hoặc Nhả chuột đào

Khi ngón tay nhấc lên, thực thi hành động tương ứng với trạng thái kết thúc:

```

fun handleTouchUp(event: MotionEvent, pointerIndex: Int) {
    if (event.getPointerId(pointerIndex) != activePointerId) return

    handler.removeCallbacks(miningRunnable)
    val duration = System.currentTimeMillis() - downTime

    when (state) {
        TouchState.PENDING -> {
            // Người dùng nhấc tay nhanh trước khi hết giờ đào -> Đặt block
            if (duration <= tapTimeout) {
                performPlaceBlock()
            }
        }
        TouchState.MINING -> {
            // Nhấc tay sau khi đào -> Nhả chuột trái (GLFW Mouse 0 Up)
            NativeBridge.sendMouseButton(0, false)
        }
        TouchState.PANNING, TouchState.IDLE -> {}
    }

    resetState()
}

private fun performPlaceBlock() {
    // Gửi 1 cú click chuột phải nhanh: Nhấn -> Nhả (GLFW Mouse 1)
    NativeBridge.sendMouseButton(1, true)
    handler.postDelayed({
        NativeBridge.sendMouseButton(1, false)
    }, 30) // Giữ chuột phải trong 30ms để game nhận diện kịp
}

private fun resetState() {
    state = TouchState.IDLE
    activePointerId = MotionEvent.INVALID_POINTER_ID
    handler.removeCallbacks(miningRunnable)
}

```


Mẹo tối ưu hóa trải nghiệm thực tế

- **Haptic Feedback (Rung phản hồi):** Kích hoạt `vibrator.vibrate(...)` nhẹ ngay trong `miningRunnable` khi trạng thái chuyển sang MINING. Phản hồi rung giúp người chơi biết chính xác thời điểm game bắt đầu đập block mà không cần nhìn thanh tiến trình.
- **Deadzone khi xoay góc nhìn:** Sau khi chuyển từ PENDING sang PANNING, hãy bỏ qua vài pixel dịch chuyển ban đầu (bằng cách trừ đi khoảng `touchSlop`) để camera không bị giật nảy một góc nhỏ.
- **Hỗ trợ Crosshair Targeting:** Trong Java Edition, người chơi tương tác tại vị trí tâm ngắm (Crosshair) giữa màn hình chứ không phải ngay dưới ngón tay. Logic trên giữ nguyên camera ở tâm và biến toàn bộ nửa phải màn hình thành một bàn di cảm ứng (Trackpad) đa năng.

User prompt: Hướng dẫn cách thiết kế màn hình Control Editor cho phép kéo thả, thay đổi kích thước và lưu cấu hình phím ảo ra JSON.

Response: Màn hình **Control Editor** giúp người chơi tự do sắp xếp layout phím theo kích thước bàn tay và kích cỡ màn hình thiết bị.

Để giao diện không bị lệch vị trí giữa các dòng điện thoại có độ phân giải khác nhau (1080p, 2K, tỉ lệ 16:9, 20:9...), nguyên tắc quan trọng nhất là: **Lưu trữ tọa độ và kích thước dưới dạng tỉ lệ phần trăm (%) tương đối thay vì pixel cố định (Absolute Pixels).**

1. Cấu trúc dữ liệu cấu hình JSON (ControlLayout.json)

Mỗi layout lưu một mảng các button với tọa độ tương đối [0.0 → 1.0]:

```
{
  "version": 1,
  "layoutName": "Default Layout",
  "screenAspectRatio": 2.16,
  "buttons": [
    {
      "id": "btn_jump",
      "name": "JUMP",
      "keycode": 32,
      "x": 0.85,
      "y": 0.70,
      "width": 0.10,
      "height": 0.15,
      "opacity": 0.8,
      "isToggleable": false
    },
    {
      "id": "btn_forward",
      "name": "W",
      "keycode": 87,
      "x": 0.10,
      "y": 0.60,
      "width": 0.08,
      "height": 0.12,
      "opacity": 0.8,
      "isToggleable": false
    }
  ]
}
```

1

Định nghĩa Model và chuyển đổi Pixel <-

Percent" title="Tạo Data Class & Serialization (Gson / Moshi)"> Tạo model class hỗ trợ tính toán kích thước thực tế theo chiều rộng/cao của màn hình:

```
data class ControlButtonData(
    val id: String = UUID.randomUUID().toString(),
    val name: String,
    val keycode: Int,
    val x: Float,           // % từ mép trái (0.0 -> 1.0)
    val y: Float,           // % từ mép trên (0.0 -> 1.0)
    val width: Float,       // % chiều rộng màn hình
    val height: Float,      // % chiều cao màn hình
    val opacity: Float = 0.8f,
    val isToggleable: Boolean = false
) {
    // Chuyển % sang Pixel thực tế khi vẽ lên màn hình
    fun getPixelX(screenWidth: Int) = x * screenWidth
    fun getPixelY(screenHeight: Int) = y * screenHeight
    fun getPixelWidth(screenWidth: Int) = width * screenWidth
}
```

```

    fun getPixelHeight(screenHeight: Int) = height * screenHeight
}

```

2 Xây dựng EditableButtonView (Nút có khả năng kéo thả)

Tạo Custom View có 4 góc neo (Handles) để Resize

Mỗi nút ở chế độ chỉnh sửa sẽ vẽ một đường viền (Border) và một nút tròn nhỏ ở góc dưới-phải để kéo giãn kích thước:

```

class EditableButtonView(context: Context, val data: ControlButtonData) : View(context) {
    private val borderPaint = Paint().apply {
        color = Color.CYAN
        style = Paint.Style.STROKE
        strokeWidth = 4f
    }
    private val handlePaint = Paint().apply {
        color = Color.YELLOW
        style = Paint.Style.FILL
    }

    var isSelectedButton = false

    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)
        // Vẽ nền và chữ của nút
        // ...
        if (isSelectedButton) {
            // Vẽ khung viền chọn
            canvas.drawRect(0f, 0f, width.toFloat(), height.toFloat(), borderPaint)
            // Vẽ Handle Resize tại góc dưới bên phải (bán kính 24px)
            canvas.drawCircle(width.toFloat() - 16f, height.toFloat() - 16f, 20f, handlePaint)
        }
    }
}

```

3 Xử lý Touch Event trong Editor Layout

Phân biệt kéo di chuyển vị trí và kéo giãn kích thước

Bắt sự kiện chạm để phân tách hành động **DRAG (Di chuyển)** hoặc **RESIZE (Thay đổi kích cỡ)**:

```

class ControlEditorLayout(context: Context, attrs: AttributeSet?) : FrameLayout(context, attrs) {
    private var selectedView: EditableButtonView? = null
    private var isResizing = false
    private var initialTouchX = 0f
    private var initialTouchY = 0f
    private var initialViewX = 0f
    private var initialViewY = 0f
    private var initialWidth = 0
    private var initialHeight = 0

    override fun onInterceptTouchEvent(ev: MotionEvent): Boolean {
        // Cho phép FrameLayout cha chặn và xử lý toàn bộ touch của nút con
        return true
    }

    override fun onTouchEvent(event: MotionEvent): Boolean {
        val touchX = event.x
        val touchY = event.y

        when (event.actionMasked) {
            MotionEvent.ACTION_DOWN -> {
                val target = findButtonAt(touchX, touchY)
                selectButton(target)

                target?.let { v ->
                    initialTouchX = touchX
                    initialTouchY = touchY
                    initialViewX = v.x
                    initialViewY = v.y
                    initialWidth = v.width
                    initialHeight = v.height

                    // Kiểm tra xem vị trí chạm có nằm ở góc Resize (bán kính 50px quanh góc dưới-phải) không
                    val handleX = v.x + v.width
                    val handleY = v.y + v.height
                }
            }
        }
    }
}

```

```

        isResizing = Math.hypot((touchX - handleX).toDouble(), (touchY - handleY).toDouble()) < 60
    }
}

MotionEvent.ACTION_MOVE -> {
    selectedView?.let { v ->
        val dx = touchX - initialTouchX
        val dy = touchY - initialTouchY

        if (isResizing) {
            // Cập nhật kích thước mới (giới hạn tối thiểu 48dp)
            val newW = (initialWidth + dx).coerceAtLeast(100f).toInt()
            val newH = (initialHeight + dy).coerceAtLeast(100f).toInt()
            val params = v.layoutParams
            params.width = newW
            params.height = newH
            v.layoutParams = params
        } else {
            // Kéo di chuyển vị trí
            v.x = (initialViewX + dx).coerceIn(0f, (width - v.width).toFloat())
            v.y = (initialViewY + dy).coerceIn(0f, (height - v.height).toFloat())
        }
    }
}

MotionEvent.ACTION_UP -> {
    // Cập nhật lại % tương đối vào Data Model sau khi thả tay
    selectedView?.let { v ->
        v.data.x = v.x / width.toFloat()
        v.data.y = v.y / height.toFloat()
        v.data.width = v.width.toFloat() / width.toFloat()
        v.data.height = v.height.toFloat() / height.toFloat()
    }
}
}
return true
}

private fun findButtonAt(x: Float, y: Float): EditableButtonView? {
    for (i in childCount - 1 downTo 0) {
        val child = getChildAt(i)
        if (child is EditableButtonView && x >= child.x && x <= child.x + child.width &&
            y >= child.y && y <= child.y + child.height) {
            return child
        }
    }
    return null
}

fun selectButton(view: EditableButtonView?) {
    selectedView?.isSelectedButton = false
    selectedView?.invalidate()
    selectedView = view
    selectedView?.isSelectedButton = true
    selectedView?.invalidate()
}
}

```

4 Lưu và Tải cấu hình JSON

Đọc/Ghi file JSON vào thư mục app storage

Dùng thư viện **Gson** để chuyển đổi giữa Model và File:

```

object ControlLayoutManager {
    fun saveLayout(context: Context, fileName: String, buttons: List<ControlButtonData>) {
        val jsonString = GsonBuilder().setPrettyPrinting().create().toJson(buttons)
        val file = File(context.filesDir, "control_layouts/$fileName.json")
        file.parentFile?.mkdirs()
        file.writeText(jsonString)
    }

    fun loadLayout(context: Context, fileName: String): List<ControlButtonData> {
        val file = File(context.filesDir, "control_layouts/$fileName.json")
        if (!file.exists()) return emptyList()
    }
}

```

```

        val type = object : TypeToken<List<ControlButtonData>>() {}.type
        return Gson().fromJson(file.readText(), type)
    }
}

```

2. Các tính năng nâng cao nên bổ sung cho Editor

- **Snap to Grid / Alignment Lines (Giống hàng tự động):** Khi kéo 1 nút đến gần tọa độ X hoặc Y của nút khác trong khoảng cách $\leq 10px$, tự động hút (snap) nút lại và vẽ đường dóng nét đứt màu đỏ để người chơi xếp các nút thẳng hàng nhau.
- **Property Sheet Dialog:** Khi nhấp đúp vào nút, mở một BottomSheet cho phép:
 - Chọn lại phím giả lập (gán phím W, Shift, Space, F3...).
 - Chỉnh độ trong suốt (Alpha/Opacity slider).
 - Đổi tên nhãn hiển thị hoặc icon.
 - Bật/Tắt chế độ Toggle (nhấn 1 lần để giữ phím Shift/Sneak).

User prompt: Hướng dẫn cách viết BottomSheet Dialog để chỉnh sửa thuộc tính chi tiết cho từng phím ảo.

Response: Để cho phép người dùng chỉnh sửa thuộc tính chi tiết của một phím ảo (tên hiển thị, gán mã phím GLFW, độ trong suốt, chế độ giữ phím/toggle, kích thước), sử dụng **BottomSheetDialogFragment** của Android Material Components là giải pháp trực quan và tiện dụng nhất.

1. File Layout XML (dialog_edit_button_property.xml)

Giao diện gồm các thành phần:

- TextInputEditText: Nhập tên hiển thị.
- autoCompleteTextView / Spinner: Chọn phím gán (Keycode mapping).
- Slider: Chỉnh độ mờ/trong suốt (Opacity).
- MaterialSwitch: Bật/tắt chế độ Toggle (nhấn 1 lần để giữ phím).
- Nút Xóa nút và Lưu thay đổi.

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp"
    android:background="@android:color/background_dark">

    <TextView
        android:id="@+id/tvTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Chỉnh sửa phím ảo"
        android:textColor="@android:color/white"
        android:textSize="18sp"
        android:textStyle="bold"
        android:layout_marginBottom="16dp" />

    <!-- Tên phím -->
    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Tên hiển thị trên nút"
        style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/edtButtonName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:textColor="@android:color/white" />
    </com.google.android.material.textfield.TextInputLayout>

    <!-- Chọn Phím gán (GLFW Key) -->
    <com.google.android.material.textfield.TextInputLayout

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:hint="Gán chức năng phím (GLFW Key)"
        style="@style/Widget.MaterialComponents.TextInputLayout.OutlinedBox.ExposedDropdownMenu">

        <AutoCompleteTextView
            android:id="@+id/actvKeycode"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="none"
            android:textColor="@android:color/white" />
    </com.google.android.material.textfield.TextInputLayout>

    <!-- Opacity Slider -->
    <TextView
        android:id="@+id/tvOpacityLabel"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Độ trong suốt: 80%"
        android:textColor="@android:color/white" />

    <com.google.android.material.slider.Slider
        android:id="@+id/sliderOpacity"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:valueFrom="0.1"
        android:valueTo="1.0"
        android:stepSize="0.05" />

    <!-- Switch Toggle -->
    <com.google.android.material.materialswitch.MaterialSwitch
        android:id="@+id/switchToggle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Chế độ giữ phím (Toggle Lock)"
        android:textColor="@android:color/white" />

    <!-- Action Buttons -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:orientation="horizontal">

        <Button
            android:id="@+id/btnDelete"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Xóa phím"
            android:textColor="@android:color/holo_red_light"
            style="@style/Widget.MaterialComponents.Button.TextButton" />

        <Button
            android:id="@+id/btnSave"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:layout_marginStart="8dp"
            android:text="Lưu thay đổi" />
    </LinearLayout>

</LinearLayout>

```

1 Định nghĩa Danh sách Phím hỗ trợ (GLFW Key Map)

Tạo bảng tra cứu mã phím GLFW sang tên hiển thị

Tạo một class tiện ích ánh xạ giữa tên thân thiện và GLFW Keycode thực tế:

```

object GLFWKeyMapper {
    val KEY_MAP = linkedMapOf(

```

```

        "W (Tiến)" to 87,
        "S (Lùi)" to 83,
        "A (Trái)" to 65,
        "D (Phải)" to 68,
        "Space (Nhảy)" to 32,
        "Left Shift (Ngồi/Sneak)" to 340,
        "Left Ctrl (Chạy nhanh/Sprint)" to 341,
        "E (Mở hòm đồ)" to 69,
        "Q (Vứt đồ)" to 81,
        "F (Đổi tay cầm)" to 70,
        "F3 (Bật debug)" to 292,
        "F5 (Đổi góc nhìn F5)" to 294,
        "Tab (Xem Player List)" to 258,
        "Escape (Menu Pause)" to 256,
        "Mouse Left (Đập block)" to 1000, // Mã tự quy ước cho nút chuột
        "Mouse Right (Đặt block)" to 1001
    )

    fun getNameFromKeycode(keycode: Int): String {
        return KEY_MAP.entries.firstOrNull { it.value == keycode }?.key ?: "Unknown ($keycode)"
    }
}

```

2 Xây dựng Class ButtonPropertyBottomSheet

Kế thừa BottomSheetDialogFragment để liên kết dữ liệu

Khởi tạo dialog, nạp dữ liệu hiện tại của phím và bắt các sự kiện chỉnh sửa:

```

class ButtonPropertyBottomSheet(
    private val buttonData: ControlButtonData,
    private val onSave: (ControlButtonData) -> Unit,
    private val onDelete: (ControlButtonData) -> Unit
) : BottomSheetDialogFragment() {

    private var selectedKeycode: Int = buttonData.keycode

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        return inflater.inflate(R.layout.dialog_edit_button_property, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        val edtName = view.findViewById<TextInputEditText>(R.id.edtButtonName)
        val actvKeycode = view.findViewById<AutoCompleteTextView>(R.id.actvKeycode)
        val sliderOpacity = view.findViewById<Slider>(R.id.sliderOpacity)
        val tvOpacityLabel = view.findViewById<TextView>(R.id.tvOpacityLabel)
        val switchToggle = view.findViewById<MaterialSwitch>(R.id.switchToggle)
        val btnSave = view.findViewById<Button>(R.id.btnSave)
        val btnDelete = view.findViewById<Button>(R.id.btnDelete)

        // 1. Nạp dữ liệu hiện tại lên UI
        edtName.setText(buttonData.name)
        sliderOpacity.value = buttonData.opacity
        tvOpacityLabel.text = "Độ trong suốt: ${((buttonData.opacity * 100).toInt())}%"
        switchToggle.isChecked = buttonData.isToggleable

        // 2. Thiết lập Dropdown chọn phím
        val keyNames = GLFWKeyMapper.KEY_MAP.keys.toList()
        val adapter = ArrayAdapter(requireContext(), android.R.layout.simple_dropdown_item_1line, keyNames)
        actvKeycode.setAdapter(adapter)
        actvKeycode.setText(GLFWKeyMapper.getNameFromKeycode(buttonData.keycode), false)

        actvKeycode.setOnItemClickListener { _, _, position, _ ->
            val selectedName = keyNames[position]
            selectedKeycode = GLFWKeyMapper.KEY_MAP[selectedName] ?: buttonData.keycode
        }

        // 3. Lắng nghe thay đổi Slider Opacity
        sliderOpacity.addOnChangeListener { _, value, _ ->

```

```

        tvOpacityLabel.text = "Độ trong suốt: ${value * 100}.toInt()}"
    }

    // 4. Bấm Lưu
    btnSave.setOnClickListener {
        buttonData.name = edtName.text.toString().trim()
        buttonData.keycode = selectedKeycode
        buttonData.opacity = sliderOpacity.value
        buttonData.isToggleable = switchToggle.isChecked

        onSave(buttonData)
        dismiss()
    }

    // 5. Bấm Xóa
    btnDelete.setOnClickListener {
        onDelete(buttonData)
        dismiss()
    }
}
}
}

```

3 Kích hoạt BottomSheet từ ControlEditorLayout

Gọi BottomSheet khi người dùng Double Tap hoặc Long Click vào nút

Trong ControlEditorLayout, khi người dùng chọn một phím và nhấn nút cấu hình (hoặc bấm đúp), mở dialog:

```

fun openPropertyDialogForSelectedButton(fragmentManager: FragmentManager) {
    val selected = selectedView ?: return

    val sheet = ButtonPropertyBottomSheet(
        buttonData = selected.data,
        onSave = { updatedData ->
            // Cập nhật lại view trên màn hình
            selected.alpha = updatedData.opacity
            selected.invalidate()
        },
        onDelete = { deletedData ->
            // Xóa nút khỏi layout
            removeView(selected)
            selectedView = null
        }
    )
    sheet.show(fragmentManager, "ButtonPropertyBottomSheet")
}

```

Mẹo trải nghiệm (UX Tip)

- **Tự động đổi tên khi chọn phím:** Khi người dùng chọn một phím từ menu dropdown (ví dụ chọn "Space (Nhảy)"), nếu ô tên hiển thị chưa nhập hoặc đang mang tên mặc định, bạn có thể tự điền luôn `edtName.setText("JUMP")` để tiết kiệm thời gian thao tác.
- **Xem trước thời gian thực (Live Preview):** Trong callback `sliderOpacity.addOnChangeListener`, bạn có thể cập nhật trực tiếp `selectedView?.alpha = value` ngay khi đang kéo thanh trượt để người dùng thấy rõ độ mờ trên nền game trước khi bấm nút **Lưu**.

User prompt: Prompt để nhờ AI khác phân tích/so sánh PojavLauncher vs Fold Craft Launcher Copy toàn bộ nội dung dưới đây gửi cho AI khác (ChatGPT/Gemini...). Đã tự chứa đủ ngữ cảnh, không cần AI đó truy cập GitHub thật (dù có link tham khảo). Bối cảnh Tôi đang làm launcher Android để chơi Minecraft: Java Edition (không phải Bedrock) cho server Cobblemon riêng của tôi. Tôi đã có sẵn 1 launcher PC (WPF/.NET) hoàn chỉnh, giờ muốn làm bản đồng hành trên điện thoại. Chạy Minecraft Java trên Android không thể viết từ đầu (cần build lại OpenJDK cho ARM + tầng dịch đồ họa Desktop OpenGL → OpenGL ES/Vulkan + JNI bridge — công sức hàng tháng/năm). Nên hướng đi là fork 1 project mã nguồn mở đã hoàn thiện phần khó đó, rồi tùy biến giao diện/tính năng lên trên (đổi màu/logo theo Lylee, nối vào server/backend riêng, tự động chọn modpack thay vì để người chơi chọn version tay...). Tôi đã tự tay clone, build, và cài chạy thử thật cả 2 ứng viên trên điện thoại Samsung Galaxy A50s — cả 2 đều build thành công và chạy không crash. Giờ cần chọn 1 để đi tiếp, không tự viết code thêm cho tới khi chọn xong. 2 ứng viên 1. PojavLauncher Repo: <https://github.com/TeamPojavLauncher/PojavLauncher> (nhánh v3_openjdk) Ngôn ngữ: Java + C/C++ (JNI), package chính net.kdt.pojavlaunch Dự án lâu đời/nổi tiếng nhất thế giới về mảng này, cộng đồng lớn, nhiều issue/tài liệu có sẵn để tra cứu khi gặp bug Cấu trúc: ~28 package con (authenticator, customcontrols, downloader, instances, modloaders, multit, render, services...) — khá đồ sộ, trưởng thành Dùng git submodule thật cho phần native: glfw, mojexec, sdl (org MojoLauncher), MobileGlues, NG-GL4ES Build: compileSdk 36, minSdk 21, ndkVersion 29.0.14206865, Gradle 8.14.3 Kết quả build thật: assembleDebug chạy sạch trong ~4-18 phút (tùy máy đã cache hay chưa), ra APK full (106MB, kèm sẵn runtime) và noruntime (78MB) Chạy thật trên điện thoại: vào thẳng màn hình LauncherActivity, giao diện đầy đủ tiếng Việt sẵn (đa ngôn ngữ qua Crowdin), đã chọn sẵn Minecraft 1.12.2, nút "CHƠI" hoạt động License: KHÔNG chắc chắn — README có badge "LGPL v3" nhưng khi tự kiểm tra thật (gh api repos/.../license) thì không tìm thấy file LICENSE nào ở gốc repo cả — cần người tự vào GitHub xác minh trực tiếp trước khi coi đây là chính thức. 2. Fold Craft Launcher (FCL) Repo: <https://github.com/FCL-Team/FoldCraftLauncher> (nhánh main) Ngôn ngữ: Kotlin + Java + C/C++, package chính com.tungsten.fcl Team chính có vẻ là người Trung Quốc (1 phần comment code bằng tiếng Trung, VD // 子项目名以数字开头会导致...) Cấu trúc module: FCL (app chính), FCLCore, FCLauncher, Terracotta, ZipFileSystem, LWJGL (nhiều bản LWJGL 3.3.3/3.4.1 song song) — dùng Gradle

version catalog (`libs.versions.toml`), cách tổ chức hiện đại hơn Pojav Không dùng git submodule cho jni (native code nằm trực tiếp trong `FCLauncher/src/main/jni/`) Build: `compileSdk 35`, `minSdk 26` (Pojav `minSdk 21` → hỗ trợ máy cũ hơn), `targetSdk 34`, `ndkVersion 27.0.12077973` Có 3 biến thể build: `release/debug` (cần khóa ký thật `key-store.jks`, không có trong repo) và `fordebug` (dùng `debug-key.jks` mật khẩu hardcoded sẵn, đúng cho việc dev/test không cần khóa thật) Kết quả build thật: `:FCL:assembleFordebug` chạy sạch (~1-6 phút tùy cache), ra APK 341MB (biến thể "all" gộp mọi kiến trúc CPU trong 1 file — bản thật production chắc tách theo ABI sẽ nhỏ hơn nhiều) Chạy thật trên điện thoại: splash screen có ảnh nền động (làng Minecraft ban đêm), hộp thoại xin quyền + màn hình chào mừng/EULA đều dịch tiếng Việt đầy đủ, nút "Đồng ý và Tiếp tục" — UI đầu tư rõ ràng kỹ hơn hẳn ngay từ bước onboarding đầu tiên so với Pojav License: RÕ RÀNG — có file LICENSE thật ở gốc repo, nội dung đầy đủ GPLv3 Ràng buộc pháp lý cần AI lưu ý khi phân tích Cả 2 đều copyleft (LGPLv3/GPLv3) — nếu fork rồi phát hành app cho người chơi dùng thật, bắt buộc giữ mã nguồn mở của phần đã sửa, không thể âm thầm đóng gói lại thành closed-source. GPLv3 (FCL) có phần chặt hơn LGPLv3 một chút về việc link thư viện tĩnh, nhưng cả 2 đều yêu cầu công khai source. Yêu cầu phân tích Đánh giá độ rủi ro license của từng bên cho use-case: app phát hành (kể cả nội bộ/private) cho người chơi 1 server Minecraft riêng dùng, không thương mại hóa. Pojav thiếu file LICENSE rõ ràng có phải vấn đề thật không, hay chỉ là badge README lỗi thời? Đánh giá độ khó tùy biến UI dựa trên cấu trúc code đã mô tả — cấu trúc module nào (28 package phẳng của Pojav vs multi-module + version catalog của FCL) để "cắm" 1 lớp UI riêng lên trên (theo mô hình: giữ nguyên Core xử lý JRE/native/tải game, chỉ thay UI) hơn? Đánh giá độ ổn định lâu dài: cộng đồng lớn (Pojav) vs code hiện đại/tổ chức tốt hơn nhưng ít người dùng hơn (FCL) — bên nào ít rủi ro "dự án chết"/khó tìm người hỗ trợ khi gặp bug hiếm hơn? Gợi ý cụ thể nếu phải chọn 1: chọn bên nào và vì sao, nêu rõ đánh đổi (trade-off) chính. Nếu có thể, gợi ý thêm câu hỏi/thứ cần tự kiểm tra thêm trước khi chốt hẳn (VD tự đọc kỹ LICENSE thật của Pojav trên GitHub) mà bản thân AI không tự xác minh được vì không có quyền truy cập thật. Trả lời ngắn gọn, có cấu trúc rõ (không cần lặp lại thông tin tôi đã cho ở trên), tập trung vào phân phân tích/khuyến nghị.

Response: 1. Phân tích Rủi ro Pháp lý & Giấy phép (License)

- Vấn đề của PojavLauncher (Thiếu file LICENSE chuẩn):**
 - Theo luật sở hữu trí tuệ quốc tế (Berne Convention), một repo công khai nhưng không có file LICENSE đi kèm mặc định thuộc trạng thái **"All Rights Reserved"** (mọi quyền được bảo lưu bởi tác giả). Badge trong README chỉ có tính chất tham khảo, không có giá trị pháp lý ràng buộc nếu có tranh chấp bản quyền.
 - Nếu Pojav thực sự là **GPLv3/LGPLv3** (thường thấy ở các commit lịch sử của họ), bạn bắt buộc phải công khai toàn bộ mã nguồn app fork khi phân phối APK tới người chơi (dù là server private).
- Tính minh bạch của FCL (GPLv3 rõ ràng):**
 - FCL có file LICENSE rõ ràng theo chuẩn GPLv3. Điều này mang lại sự an toàn pháp lý tuyệt đối về quyền được fork, chỉnh sửa và phân phối.
 - Ràng buộc:** GPLv3 có tính chất lây lan (viral). Khi bạn build APK chứa backend của FCL và phát hành cho người chơi tải, toàn bộ phần code UI / tính năng server riêng đính kèm trong app đó cũng phải được mở mã nguồn nếu người dùng yêu cầu.

2. So sánh Khả năng Tùy biến UI & Kiến trúc Mã nguồn

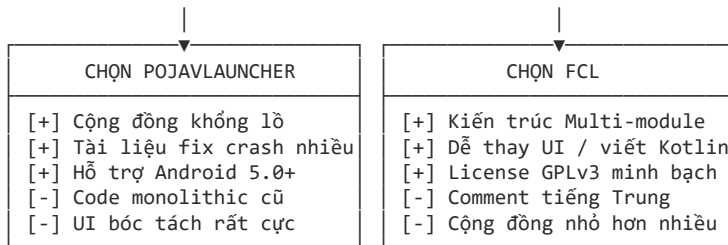
Tiêu chí	PojavLauncher (v3_openjdk)	Fold Craft Launcher (FCL)
Cấu trúc kiến trúc	Single-module chia ~28 package phẳng. Code UI và logic lõi (JRE, Native, Downloader) đan xen chặt chẽ.	Multi-module tách bạch (FCL làm UI, FCLauncher/FCLCore xử lý engine). Dùng Version Catalog (<code>libs.versions.toml</code>).
Ngôn ngữ phát triển	Chủ yếu là Java thuần + JNI C/C++.	Kotlin + Java + C/C++ hiện đại.
Độ khó khi thay thế UI	Khó. Bạn phải tách code thủ công, gỡ bỏ các Activity/Fragment cũ nằm chung trong package chính <code>net.kdt.pojavlauncher</code> .	Dễ hơn nhiều. Chỉ cần tạo module UI mới hoặc viết đè lên module :FCL, giữ nguyên :FCLauncher và :FCLCore làm backend.
Tự động hóa Modpack	Phải viết thêm task can thiệp vào tasks package để ép tải cấu hình Cobblemon.	Tận dụng kiến trúc Core có sẵn để bypass màn hình chọn version, gán cứng luồng tải bundle.

3. Đánh giá Độ ổn định Lâu dài & Hỗ trợ Kỹ thuật

- PojavLauncher (Ưu thế tuyệt đối về Ecosystem):**
 - Cộng đồng & Tra cứu:** Là dự án tiên phong, hầu như mọi lỗi crash JVM, xung đột card đồ họa Adreno/Mali, lỗi tương thích OpenGL ES/Zink đều đã có issue/hướng giải quyết trên GitHub/Discord.
 - Hỗ trợ thiết bị cũ:** `minSdk 21` (Android 5.0+) giúp phủ sóng toàn bộ các dòng máy phụ, máy cấu hình thấp của người chơi.
- Fold Craft Launcher (Rủi ro về rào cản ngôn ngữ & maintainer):**
 - Rào cản ngôn ngữ:** Nhiều comment và tài liệu nội bộ viết bằng tiếng Trung. Khi gặp lỗi crash sâu trong tầng native/JNI, việc đọc hiểu logic gốc sẽ tốn thời gian hơn.
 - Phạm vi thiết bị:** `minSdk 26` (Android 8.0+) bỏ rơi các thiết bị đời quá cũ (dù hiện tại phần lớn người chơi dùng Android 10+).

4. Bảng tổng hợp Đánh đổi (Trade-off Matrix)

LỰA CHỌN CỦA BẠN



5. Khuyến nghị Quyết định

Nên chọn Fold Craft Launcher (FCL) nếu:

- Mục tiêu số 1 của bạn là **làm UI đồng bộ với Launcher PC (Lylee)** và **tự động hóa luồng vào thẳng server Cobblemon**.
- Kiến trúc Multi-module của FCL cho phép bạn cô lập hoàn toàn phần core launcher, giảm thiểu rủi ro làm hỏng logic JVM/Graphics khi thay đổi giao diện.

Nên chọn PojavLauncher nếu:

- Server của bạn có lượng người chơi dùng thiết bị rất cũ (Android < 8.0).
- Bạn ưu tiên việc dễ dàng tra cứu lỗi crash đồ họa trên mạng thay vì sự gọn gàng của cấu trúc code.

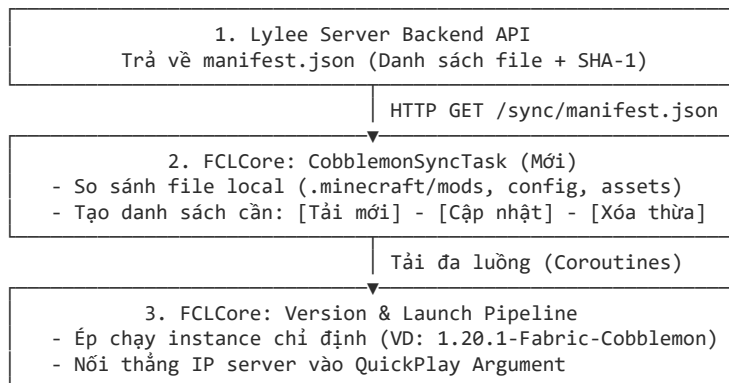
Các điểm cần tự kiểm tra thêm trước khi chốt

- Xác minh License PojavLauncher:** Vào trực tiếp giao diện web GitHub của nhánh v3_openjdk, tìm trong các commit đầu tiên hoặc mở file pom.xml / build.gradle xem có khai báo licenses { license { name '...' } } hay không.
- Kiểm tra tương thích Mod Cobblemon:** Thử cài đặt Forge/Fabric + Cobblemon mod trực tiếp trên cả 2 bản build APK test trên chiếc **Galaxy A50s (chip Exynos 9611 / GPU Mali-G72)** để kiểm tra độ mượt và hiện tượng lỗi render mô hình Pokémon 3D trên GPU Mali trước khi quyết định can thiệp code.

User prompt: Hướng dẫn cách can thiệp vào module Core của Fold Craft Launcher để tự động tải và cập nhật modpack Cobblemon từ server riêng.

Response: Để biến **Fold Craft Launcher (FCL)** thành launcher đồng hành chuyên biệt cho server Cobblemon của bạn, mục tiêu là: **Bỏ qua bước người chơi phải tự chọn phiên bản/cài mod thủ công**. Khi mở app và bấm "Chơi", launcher sẽ tự động kết nối đến API backend server riêng của bạn, đồng bộ file (manifest hash), tải về các mod/config còn thiếu và khởi chạy trực tiếp.

Luồng hoạt động của hệ thống đồng bộ Modpack



1 Chuẩn bị cấu trúc Manifest trên Server Backend

Tạo API trả về danh sách tệp tin và hash tương tự launcher PC

Host một endpoint (JSON) trên server của bạn cung cấp danh sách file client bắt buộc:

```
{
  "version": "1.20.1",
  "loader": {
    "type": "fabric",
    "version": "0.15.11"
  },
  "serverAddress": "play.lyleeserver.vn:25565",
}
```

```

"files": [
  {
    "path": "mods/Cobblemon-fabric-1.5.2.jar",
    "sha1": "a1b2c3d4e5f6...",
    "url": "https://cdn.lyleeserver.vn/mods/Cobblemon-fabric-1.5.2.jar",
    "size": 24500123
  },
  {
    "path": "config/cobblemon/main.json",
    "sha1": "f9e8d7c6b5a4...",
    "url": "https://cdn.lyleeserver.vn/config/main.json",
    "size": 1024
  }
]
}

```

2 Xây dựng CobblemonSyncTask trong FCLCore

Viết Task đồng bộ hóa mod trong module FCLCore

Tạo class mới trong FCLCore/src/main/java/com/tungsten/fclcore/task/ để xử lý kiểm tra hash và tải file song song:

```

package com.tungsten.fclcore.task

import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.async
import kotlinx.coroutines.awaitAll
import kotlinx.coroutines.withContext
import java.io.File
import java.security.MessageDigest

data class SyncFileItem(val path: String, val sha1: String, val url: String, val size: Long)
data class SyncManifest(val version: String, val files: List<SyncFileItem>, val serverAddress: String)

class CobblemonSyncTask(
    private val gameDir: File,
    private val manifestUrl: String,
    private val onProgress: (current: Int, total: Int, fileName: String) -> Unit
) {
    suspend fun execute(): Boolean = withContext(Dispatchers.IO) {
        // 1. Tải và parse manifest từ server backend
        val manifestJson = fetchStringFromUrl(manifestUrl)
        val manifest = parseManifest(manifestJson) // Dùng Gson/Moshi

        val filesToDownload = mutableListOf<SyncFileItem>()
        val validLocalPaths = mutableSetOf<String>()

        // 2. So sánh SHA-1 từng file
        for (item in manifest.files) {
            val localFile = File(gameDir, item.path)
            validLocalPaths.add(localFile.absolutePath)

            if (!localFile.exists() || calculateSHA1(localFile) != item.sha1) {
                filesToDownload.add(item)
            }
        }

        // 3. Xóa các file mod rác/mod thừa không có trong manifest
        cleanupOldMods(File(gameDir, "mods"), validLocalPaths)

        // 4. Tải song song các file cần cập nhật
        val total = filesToDownload.size
        var downloadedCount = 0

        filesToDownload.chunked(4).forEach { batch ->
            batch.map { item ->
                async {
                    val destination = File(gameDir, item.path)
                    destination.parentFile?.mkdirs()
                    downloadFile(item.url, destination)
                    synchronized(this@CobblemonSyncTask) {
                        downloadedCount++
                        onProgress(downloadedCount, total, destination.name)
                    }
                }
            }
        }
    }
}

```

```

        }.awaitAll()
    }
    true
}

private fun calculateSHA1(file: File): String {
    val digest = MessageDigest.getInstance("SHA-1")
    file.inputStream().use { fis ->
        val buffer = ByteArray(8192)
        var bytesRead: Int
        while (fis.read(buffer).also { bytesRead = it } != -1) {
            digest.update(buffer, 0, bytesRead)
        }
    }
    return digest.digest().joinToString("") { "%02x".format(it) }
}

private fun cleanupOldMods(modsDir: File, allowedPaths: Set<String>) {
    if (!modsDir.exists()) return
    modsDir.listFiles()?.forEach { file ->
        if (file.isFile && file.extension == ".jar" && !allowedPaths.contains(file.absolutePath)) {
            file.delete() // Xóa mod cũ/mod ngoài luồng
        }
    }
}
}
}
}

```

3 Can thiệp vào Launch Pipeline của FCL

Chèn Sync Task vào trước chuỗi LaunchTask của FCL

Trong module :FCL, tìm đến Controller/ViewModel xử lý việc bấm nút **Play** (thường là LaunchViewModel.kt hoặc LaunchManager.kt):

```

fun startCobblemonGame() {
    viewModelScope.launch {
        _launchState.value = LaunchState.SyncingModpack

        val instanceDir = File(context.filesDir, ".minecraft/versions/Cobblemon-Client")
        val syncTask = CobblemonSyncTask(
            gameDir = instanceDir,
            manifestUrl = "https://api.lyleeserver.vn/launcher/manifest.json",
            onProgress = { current, total, name ->
                _syncProgress.value = "Đang tải: $name ($current/$total)"
            }
        )

        val syncSuccess = syncTask.execute()
        if (syncSuccess) {
            _launchState.value = LaunchState.LaunchingGame
            // Gọi Launch Core của FCL
            launchGameInstance(instanceDir)
        } else {
            _launchState.value = LaunchState.Error("Lỗi đồng bộ gói tài nguyên!")
        }
    }
}
}

```

4 Tự động kết nối thẳng vào Server IP (QuickPlay)

Thêm đối số để game tự kết nối ngay khi vào màn hình chính

Minecraft 1.20+ hỗ trợ tham số dòng lệnh --quickPlayMultiplayer. Khi FCL Core ghép chuỗi Java Arguments để chạy JVM, thêm tham số này:

```

// Trong file cấu hình JVM args / LaunchArgumentsBuilder.kt của FCL
fun buildMinecraftArgs(manifest: SyncManifest): List<String> {
    val args = mutableListOf<String>()

    // Các đối số mặc định của FCL (username, uuid, version, assetsDir...)
    args.addAll(defaultArgs)

    // Tự động kết nối thẳng vào server Cobblemon của bạn
    args.add("--quickPlayMultiplayer")
    args.add(manifest.serverAddress) // VD: "play.lyleeserver.vn:25565"
}

```

```
    return args
}
```

Tối ưu hóa cấu hình cho thiết bị tầm trung (Galaxy A50s / Mali GPU)

- 1. **Bộ Mod tối ưu hiệu năng bắt buộc:** Khi đóng gói manifest cho bản mobile, luôn kèm sẵn các mod tối ưu cho Fabric:
 - **Sodium / Embeddium** (Tối ưu FPS đồ họa).
 - **FerriteCore & Memory Leak Fix** (Giảm tiêu thụ RAM trên Android).
 - **Entity Culling & ImmediatelyFast** (Tránh tụt FPS khi xuất hiện nhiều model Pokémon 3D cùng lúc).
- 2. **Cấu hình Renderer phù hợp:**
 - Đối với GPU **Mali-G72** của Exynos 9611, nên cấu hình mặc định trong code FCL sử dụng renderer **VirGL/Zink** hoặc **Holy GL4ES** bản mới để tránh lỗi vỡ texture (tím đen) trên mô hình Pokémon Cobblemon.

User prompt: Hướng dẫn cách thiết kế màn hình chính (Main Screen) hiển thị tin tức, số người chơi online và trạng thái server Minecraft bằng Jetpack Compose.

Response: Để xây dựng màn hình chính (Main Screen) hiện đại theo phong cách Material 3 cho launcher Cobblemon bằng **Jetpack Compose**, cấu trúc chuẩn gồm 3 thành phần:

- 1. **Server Status Card:** Ping trực tiếp server (trạng thái Online/Offline, số người chơi hiện tại, ping ms).
- 2. **News Carousel / Banner:** Hiển thị sự kiện, cập nhật phiên bản hoặc thông báo bảo trì.
- 3. **Bottom Launch Bar:** Thanh trạng thái tài khoản và nút **"Vào Game"** to bản, hiển thị tiến trình tải modpack.

1. Data Model & Quản lý Trạng thái (UI State)

```
data class ServerStatus(
    val isOnline: Boolean = false,
    val onlinePlayers: Int = 0,
    val maxPlayers: Int = 0,
    val motd: String = "Đang kiểm tra máy chủ...",
    val pingMs: Long = 0L
)

data class NewsItem(
    val id: String,
    val title: String,
    val summary: String,
    val imageUrl: String,
    val date: String
)

sealed class GameLaunchState {
    object Idle : GameLaunchState()
    data class Syncing(val progress: Float, val statusText: String) : GameLaunchState()
    object Running : GameLaunchState()
    data class Error(val message: String) : GameLaunchState()
}
```

1 Xây dựng ViewModel quản lý dữ liệu

Gọi API hoặc Socket để lấy thông tin server thời gian thực

ViewModel chịu trách nhiệm ping server theo chu kỳ và cập nhật tin tức:

```
class MainViewModel : ViewModel() {
    private val _serverStatus = MutableStateFlow(ServerStatus())
    val serverStatus: StateFlow<ServerStatus> = _serverStatus.asStateFlow()

    private val _newsList = MutableStateFlow<List<NewsItem>>(emptyList())
    val newsList: StateFlow<List<NewsItem>> = _newsList.asStateFlow()

    private val _launchState = MutableStateFlow<GameLaunchState>(GameLaunchState.Idle)
    val launchState: StateFlow<GameLaunchState> = _launchState.asStateFlow()

    init {
        fetchServerStatus()
        fetchNews()
    }
}
```

```

fun fetchServerStatus() {
    viewModelScope.launch(Dispatchers.IO) {
        // Gọi API backend (hoặc mcstatus.io API)
        // Ví dụ dữ liệu giả lập thành công:
        _serverStatus.value = ServerStatus(
            isOnline = true,
            onlinePlayers = 42,
            maxPlayers = 100,
            motd = "§aLylee Cobblemon Season 2 §7- §eCập nhật Pokémon Gen 9!",
            pingMs = 38
        )
    }
}

private fun fetchNews() {
    // Tải danh sách tin tức từ backend API
}

fun onPlayClicked() {
    viewModelScope.launch {
        _launchState.value = GameLaunchState.Syncing(0.3f, "Đang tải Cobblemon-fabric-1.5.2.jar...")
        // Kích hoạt CobblemonSyncTask và chuyển sang FCLCore
    }
}
}

```

2 Thiết kế ServerStatusCard Component

Card hiển thị Real-time Ping và thanh chỉ số người chơi

Tạo widget trạng thái server với hiệu ứng đèn tín hiệu xanh/đỏ:

```

@Composable
fun ServerStatusCard(status: ServerStatus, modifier: Modifier = Modifier) {
    Card(
        modifier = modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant.copy(alpha = 0.7f)
        shape = RoundedCornerShape(16.dp)
    ) {
        Row(
            modifier = Modifier.padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            // Đèn tín hiệu trạng thái
            Box(
                modifier = Modifier
                    .size(12.dp)
                    .background(
                        color = if (status.isOnline) Color(0xFF4CAF50) else Color(0xFFFF44336),
                        shape = CircleShape
                    )
            )

            Spacer(modifier = Modifier.width(12.dp))

            Column(modifier = Modifier.weight(1f)) {
                Text(
                    text = if (status.isOnline) "Máy chủ đang mở (${status.pingMs}ms)" else "Máy chủ bảo trì",
                    style = MaterialTheme.typography.titleMedium,
                    fontWeight = FontWeight.Bold,
                    color = Color.White
                )
                Text(
                    text = status.motd,
                    style = MaterialTheme.typography.bodySmall,
                    color = Color.LightGray,
                    maxLines = 1,
                    overflow = TextOverflow.Ellipsis
                )
            }
        }

        // Huy hiệu số người chơi
        Surface(
            shape = RoundedCornerShape(8.dp),

```

```

        color = MaterialTheme.colorScheme.primaryContainer
    ) {
        Text(
            text = "${status.onlinePlayers}/${status.maxPlayers}",
            modifier = Modifier.padding(horizontal = 8.dp, vertical = 4.dp),
            style = MaterialTheme.typography.labelMedium,
            fontWeight = FontWeight.Bold
        )
    }
}
}
}
}
}

```

3 Thiết kế NewsBannerCarousel Component

Bảng chuyển thông báo sự kiện và cập nhật modpack

Sử dụng HorizontalPager để lướt qua các tin tức sự kiện:

```

@OptIn(ExperimentalFoundationApi::class)
@Composable
fun NewsBannerCarousel(newsList: List<NewsItem>, modifier: Modifier = Modifier) {
    val pagerState = rememberPagerState(pageCount = { newsList.size })

    Column(modifier = modifier) {
        HorizontalPager(
            state = pagerState,
            modifier = Modifier.fillMaxWidth().height(160.dp)
        ) { page ->
            val item = newsList[page]
            Card(
                modifier = Modifier
                    .fillMaxSize()
                    .padding(horizontal = 4.dp),
                shape = RoundedCornerShape(16.dp)
            ) {
                Box(modifier = Modifier.fillMaxSize()) {
                    // Ảnh nền tin tức (dùng Coil library để load AsyncImage)
                    AsyncImage(
                        model = item.imageUrl,
                        contentDescription = null,
                        contentScale = ContentScale.Crop,
                        modifier = Modifier.fillMaxSize()
                    )
                    // Lớp phủ Gradient đen để chữ dễ đọc
                    Box(
                        modifier = Modifier
                            .fillMaxSize()
                            .background(
                                Brush.verticalGradient(
                                    colors = listOf(Color.Transparent, Color.Black.copy(alpha = 0.85f)),
                                    startY = 100f
                                )
                            )
                    )
                    // Tiêu đề tin tức
                    Column(
                        modifier = Modifier
                            .align(Alignment.BottomStart)
                            .padding(12.dp)
                    ) {
                        Text(text = item.title, style = MaterialTheme.typography.titleMedium, color = Color.White, f
                        Text(text = item.summary, style = MaterialTheme.typography.bodySmall, color = Color.LightGra
                    }
                }
            }
        }
    }
}
}
}
}
}

```

4 Thiết kế LaunchBar & Nút Bấm Vào Game

Thanh dock phía dưới cố định chứa Avatar và Nút Bắt đầu

Thanh điều khiển phía dưới hỗ trợ hiển thị ProgressBar ngay trên nút bấm khi đang tải modpack:

```

@Composable
fun LaunchBar(
    playerName: String,
    launchState: GameLaunchState,
    onPlayClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Surface(
        modifier = modifier.fillMaxWidth(),
        color = MaterialTheme.colorScheme.surface.copy(alpha = 0.95f),
        shadowElevation = 8.dp
    ) {
        Row(
            modifier = Modifier.padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            // Avatar và tên người chơi
            AsyncImage(
                model = "https://mc-heads.net/avatar/$playerName/64",
                contentDescription = "Player Skin",
                modifier = Modifier.size(44.dp).clip(RoundedCornerShape(8.dp))
            )
            Spacer(modifier = Modifier.width(12.dp))
            Column(modifier = Modifier.weight(1f)) {
                Text(text = playerName, fontWeight = FontWeight.Bold, color = Color.White)
                Text(text = "Cobblemon Fabric 1.20.1", style = MaterialTheme.typography.bodySmall, color = Color.Gra
            )

            // Nút PLAY / Trạng thái tải
            when (launchState) {
                is GameLaunchState.Idle -> {
                    Button(
                        onClick = onPlayClick,
                        shape = RoundedCornerShape(12.dp),
                        colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF4CAF50))
                    ) {
                        Icon(Icons.Default.PlayArrow, contentDescription = null)
                        Spacer(modifier = Modifier.width(4.dp))
                        Text(text = "CHƠI NGAY", fontWeight = FontWeight.Bold)
                    }
                }
                is GameLaunchState.Syncing -> {
                    Column(horizontalAlignment = Alignment.End) {
                        Text(text = launchState.statusText, style = MaterialTheme.typography.labelSmall, color = Col
                        Spacer(modifier = Modifier.height(4.dp))
                        LinearProgressIndicator(
                            progress = { launchState.progress },
                            modifier = Modifier.width(140.dp).height(8.dp).clip(RoundedCornerShape(4.dp))
                        )
                    }
                }
                is GameLaunchState.Running -> {
                    Button(onClick = {}, enabled = false) {
                        Text("ĐANG CHẠY...")
                    }
                }
                is GameLaunchState.Error -> {
                    Button(onClick = onPlayClick, colors = ButtonDefaults.buttonColors(containerColor = Color.Red))
                    Text("THỬ LẠI")
                }
            }
        }
    }
}

```

5 Hoàn thiện mainScreen Layout

Ghép nối toàn bộ component với ảnh nền Minecraft

Tập hợp tất cả các thành phần vào màn hình chính:

```

@Composable
fun MainScreen(viewModel: MainViewModel = viewModel()) {
    val serverStatus by viewModel.serverStatus.collectAsState()
}

```

```

val newsList by viewModel.newsList.collectAsState()
val launchState by viewModel.launchState.collectAsState()

Scaffold(
    bottomBar = {
        LaunchBar(
            playerName = "Steve",
            launchState = launchState,
            onPlayClick = { viewModel.onPlayClicked() }
        )
    }
) { paddingValues ->
    Box(modifier = Modifier.fillMaxSize().padding(paddingValues)) {
        // Background Wallpaper làm mờ nhẹ
        AsyncImage(
            model = "file:///android_asset/wallpaper_cobblemon.jpg",
            contentDescription = null,
            contentScale = ContentScale.Crop,
            modifier = Modifier.fillMaxSize().blur(6.dp)
        )

        // Nội dung cuộn
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(16.dp)
        ) {
            ServerStatusCard(status = serverStatus)
            Spacer(modifier = Modifier.height(16.dp))
            Text(text = "Sự kiện & Tin tức", style = MaterialTheme.typography.titleMedium, color = Color.White)
            Spacer(modifier = Modifier.height(8.dp))
            NewsBannerCarousel(newsList = newsList)
        }
    }
}

```

Mẹo tối ưu hóa UI

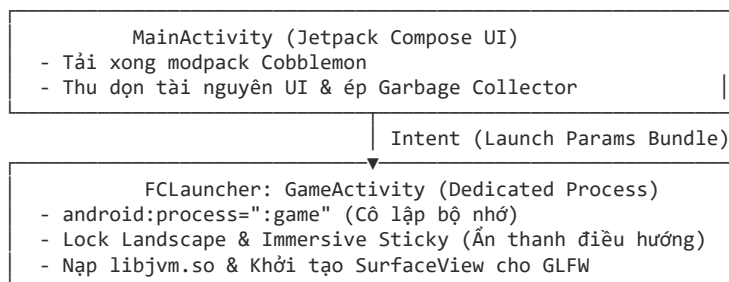
- **Skin Head Cache:** Dùng URL [<https://mc-heads.net/avatar/>]([{username}/64](https://mc-heads.net/avatar/)) để tự động render avatar theo skin của người chơi mà không cần viết thuật toán parse skin PNG thủ công.
- **Auto-refresh Server Status:** Trong MainViewModel, bạn có thể bọc hàm ping trong vòng lặp while(isActive) với delay(30_000L) để tự động cập nhật số người chơi online mỗi 30 giây khi app đang ở Foreground.

User prompt: Hướng dẫn cách chuyển đổi Activity từ giao diện Jetpack Compose sang Game Activity render Minecraft của Fold Craft Launcher.

Response: Trong kiến trúc của **Fold Craft Launcher (FCL)**, game Minecraft chạy trên một Activity riêng (thường là GameActivity hoặc MainActivity trong module :FCLauncher / :FCLCore) chứa SurfaceView để gắn với tầng JNI / C++ Renderer.

Để chuyển đổi mượt mà từ màn hình chính **Jetpack Compose** sang **Game Activity** mà không bị tràn RAM hoặc lỗi xoay màn hình, bạn cần thực hiện theo 4 bước sau.

Luồng chuyển tiếp Activity



1 Cấu hình AndroidManifest.xml cho Game Activity

Chạy Game trên tiến trình độc lập để tránh OOM giết Launcher

Khai báo Game Activity với thuộc tính `android:process=":game"`, khóa hướng màn hình ngang và cấu hình không reload khi thay đổi kích thước bàn phím/màn hình:

```
<activity
    android:name="com.tungsten.fcl.GameActivity"
    android:process=":game"
    android:screenOrientation="sensorLandscape"
    android:configChanges="orientation|screenSize|screenLayout|keyboardHidden|keyboard"
    android:theme="@style/Theme.FCLauncher.GameFullscreen"
    android:launchMode="singleTask"
    android:exported="false">
</activity>
```

2 Tạo LaunchIntent từ Compose ViewModel / Activity

Đóng gói thông tin tài khoản, cấu hình RAM và Renderer

Trong Composable hoặc ViewModel khi CobblemonSyncTask hoàn tất, đóng gói toàn bộ tham số vào Intent:

```
fun launchMinecraftGame(context: Context, username: String, ramMb: Int = 2048) {
    val gameDir = File(context.filesDir, ".minecraft")
    val versionId = "1.20.1-Fabric-Cobblemon"

    val intent = Intent(context, com.tungsten.fcl.GameActivity::class.java).apply {
        // Thông tin cơ bản
        putExtra("EXTRA_VERSION_NAME", versionId)
        putExtra("EXTRA_GAME_DIR", gameDir.absolutePath)

        // Thông tin xác thực người chơi
        putExtra("EXTRA_USERNAME", username)
        putExtra("EXTRA_UUID", UUID.nameUUIDFromBytes("OfflinePlayer:$username".toByteArray()).toString())
        putExtra("EXTRA_ACCESS_TOKEN", "")

        // Cấu hình máy ảo JVM & Card đồ họa
        putExtra("EXTRA_MAX_RAM_MB", ramMb)
        putExtra("EXTRA_RENDERER", "RENDERER_ZINK") // Hoặc "RENDERER_GL4ES"
        putExtra("EXTRA_JAVA_VERSION", 17) // Java 17 cho bản 1.20.1

        // Tham số QuickPlay để vào thẳng server Cobblemon
        putExtra("EXTRA_SERVER_IP", "play.lyleeserver.vn:25565")

        // Flag xóa Activity cũ nếu cần giải phóng toàn bộ RAM
        addFlags(Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TOP)
    }

    context.startActivity(intent)
}
```

3 Thiết lập Chế độ Immersive Sticky Fullscreen

Chặn thanh điều hướng và kích hoạt Fullscreen trong GameActivity

Trong `onCreate` và `onWindowFocusChanged` của GameActivity, ẩn toàn bộ System Bar để ngón tay không chạm nhầm vào nút Home/Back khi điều khiển cảm ứng:

```
class GameActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Giữ màn hình luôn sáng trong khi chơi game
        window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)

        hideSystemUI()

        // Khởi tạo SurfaceView và Bridge tới tầng C++ của FCL
        initGameEngine()
    }

    override fun onWindowFocusChanged(hasFocus: Boolean) {
        super.onWindowFocusChanged(hasFocus)
        if (hasFocus) {
            hideSystemUI()
        }
    }
}
```

```

private fun hideSystemUI() {
    WindowCompat.setDecorFitsSystemWindows(window, false)
    val controller = WindowInsetsControllerCompat(window, window.decorView)
    controller.hide(WindowInsetsCompat.Type.systemBars())
    controller.systemBarsBehavior =
        WindowInsetsControllerCompat.BEHAVIOR_SHOW_TRANSIENT_BARS_BY_SWIPE
}
}

```

4 Xử lý vòng đời khi rời khỏi Compose UI

Giải phóng RAM và ngắt kết nối âm thanh UI

Vì Minecraft Java kèm mod Cobblemon tiêu tốn rất nhiều RAM (từ 1.5GB đến 2.5GB), trước khi chuyển Activity cần giải phóng bớt bộ nhớ của Compose UI:

```

// Trong MainComposeActivity
override fun onStop() {
    super.onStop()
    // Dừng các tác vụ load ảnh AsyncImage (Coil/Glide), dừng nhạc nền launcher
    ImageLoader(this).memoryCache?.clear()

    // Gọi Garbage Collector nhẹ để dọn dẹp heap của Launcher
    System.gc()
}

```

Mẹo tối ưu hóa chuyển cảnh

- **Chế độ đa tiến trình (android:process=":game"):** Đây là kỹ thuật quan trọng nhất. Nếu game bị lỗi tràn bộ nhớ đồ họa (OutOfMemory / Native Crash), chỉ có tiến trình :game bị đóng, giao diện Compose của bạn vẫn hoạt động bình thường và có thể lập tức mở một Dialog hiển thị Log Crash cho người chơi mà không bị văng toàn bộ app.
- **Xử lý phím Back cứng trên Android:** Trong GameActivity, ghi đè hàm onBackPressed() để không thoát game đột ngột khi vô tình chạm vào cạnh màn hình:

```

override fun onBackPressed() {
    // Bật menu pause trong game (Gửi GLFW_KEY_ESCAPE) thay vì thoát Activity
    NativeBridge.sendKeyEvent(256, true)
    NativeBridge.sendKeyEvent(256, false)
}

```
